FINAL YEAR PROJECT PROPOSAL

Department of Computer Science

National University of Computer and Emerging Science

PROJECT A.E.G.I.S.

Adaptive Execution and Goal-oriented Intelligence System

Project Team Members

Hira Khalid - 23L-2594

Doureesha Batool - 23L-2651

Hashir Rauf - 23L-2572

Domain: Artificial Intelligence · Human-Computer Interaction · Agentic AI · NLP · Operating Systems

Contents

1. Abstract

Modern computing environments demand users to simultaneously manage files, applications, browser sessions, reminders, and multifaceted workflows across fragmented interfaces. Despite substantial advances in operating system capabilities, interaction paradigms have remained largely reactive and command-driven, imposing considerable cognitive burdens on users. Contemporary AI-powered virtual assistants-including Apple Siri, Amazon Alexa, and Google Assistant-execute explicit user commands but lack the contextual depth necessary to function as genuine cognitive partners. They do not model user intent across sessions, cannot reason about ongoing desktop activity, and are incapable of proactive, multi-step task planning.

This proposal presents AEGIS (Adaptive Execution and Goal-oriented Intelligence System), an AI-powered desktop copilot designed to serve as an intelligent intermediary between the user and the operating system. AEGIS integrates natural language understanding, desktop context retrieval, long-term episodic memory, and agentic task planning to reduce cognitive load and streamline complex multi-application workflows. Unlike existing assistants, AEGIS does not merely respond to commands; it continuously maintains an awareness of the user's current environment-open applications, recent files, browser activity, and interaction history-to proactively infer intent and orchestrate goal-directed actions.

The proposed system is grounded in recent advances in Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), multi-step agentic planning, and human-computer interaction research. The research contribution of this project is threefold: (1) the design of a context-aware desktop intelligence framework that fuses real-time OS context with LLM-based reasoning; (2) an empirical evaluation of cognitive load reduction using established psychometric instruments such as the NASA Task Load Index (NASA-TLX); and (3) a safety-conscious human-in-the-loop control architecture for desktop automation agents. Evaluation will be conducted through controlled user studies measuring task completion time, interaction count, cognitive workload, and task success rate against baseline traditional OS interactions.

2. Background and Motivation

2.1 The Fragmentation of Digital Work

The contemporary digital workspace is characterised by profound fragmentation. Knowledge workers routinely operate across dozens of applications-word processors, email clients, web browsers, project management tools, code editors, and communication platforms-without cohesive contextual continuity between them. Research by González and Mark (2004) established that knowledge workers switch tasks approximately every three minutes and experience significant recovery costs when their focus is interrupted. More recent work by Gonzalez and Mark (2022)

confirms that fragmented attention patterns persist and are exacerbated by the proliferation of digital notification channels.

The human cost of this fragmentation is well documented. Users expend cognitive resources not on substantive work but on meta-tasks: locating files, reconstructing prior work context, switching between applications, and repeating routine operations. These meta-tasks represent unproductive overhead that intelligent automation could substantially reduce.

2.2 Limitations of Current AI Assistants

The first generation of AI-powered desktop assistants-Cortana, Siri, Google Assistant-represented a meaningful step towards natural language interaction with computing environments. However, these systems share fundamental architectural limitations that constrain their utility as cognitive partners:

- Reactive Command Execution: These assistants respond to explicit, fully specified commands and cannot infer partially stated intent from contextual signals.
- Shallow Contextual Awareness: They do not model the user's current desktop state, open documents, or recent activity to disambiguate requests or proactively offer assistance.
- Absence of Long-Term Memory: Session boundaries erase all prior interaction history, preventing personalisation and learning over time.
- Lack of Multi-Step Planning: They cannot decompose high-level user goals into sequences of actions spanning multiple applications.
- Limited Desktop Integration: Most assistants operate primarily in the voice-input/text-output modality without genuine integration into the file system, running processes, or application state.

2.3 The Emergence of Agentic AI Systems

The advent of large language models (LLMs) capable of chain-of-thought reasoning, tool use, and multi-step planning has created new possibilities for AI-assisted computing. Systems such as AutoGPT, BabyAGI, LangChain agents, and more recently OpenAI Assistants with Code Interpreter have demonstrated that LLMs can function as planning engines capable of decomposing complex goals into executable sub-tasks (Yao et al., 2023; Wang et al., 2023). Concurrently, Retrieval-Augmented Generation (RAG) has emerged as a powerful paradigm for grounding LLM responses in user-specific contextual knowledge (Lewis et al., 2020; Gao et al., 2023).

These developments collectively establish the technical foundation for a new class of desktop assistant that transcends command-response interaction. AEGIS is positioned at the intersection of these emerging paradigms, applying agentic LLM planning, contextual retrieval, and desktop automation to the problem of intelligent OS-level task management.

3. Problem Statement

Despite the existence of increasingly powerful operating systems and a growing ecosystem of AI-powered tools, users continue to bear the primary cognitive burden of managing their digital environments. The fundamental problem is the absence of an intelligent system that can:

1. Accurately infer user intent from natural language input grounded in real-time desktop context.
2. Retrieve and synthesise relevant contextual information from across the user's digital environment-including the file system, open applications, browser activity, and interaction history-to inform task planning.
3. Autonomously plan and execute multi-step goal-directed tasks across heterogeneous applications while maintaining user safety and control.
4. Maintain persistent, personalised memory of user preferences, recurring workflows, and prior interactions to provide increasingly tailored assistance over time.
5. Measurably reduce the cognitive load and productivity overhead associated with routine desktop task management.

The core research problem may therefore be formally stated as follows: How can an AI-powered operating system copilot accurately understand user intent, retrieve and leverage contextual information from the desktop environment, and autonomously execute goal-directed tasks while reducing cognitive load, minimising context-switching overhead, and preserving user safety and control?

4. Research Gap

A systematic review of the existing literature reveals several significant gaps that AEGIS is positioned to address:

G1 - Absence of Holistic Desktop Context Integration: Existing work on LLM-based desktop automation (e.g., UFO, AppAgent, OSWorld) focuses primarily on GUI element identification and single-application task execution. No existing system integrates multi-source desktop context-file system state, open processes, browser activity, clipboard history, and interaction memory-as a unified retrieval corpus for intent disambiguation and task planning.

G2 - Lack of Persistent Cross-Session Memory in Desktop Agents: Current agentic frameworks such as LangChain, AutoGen, and CrewAI do not provide persistent, user-specific episodic memory that accumulates across sessions. This limits personalisation and prevents the system from learning recurring workflows or user preferences over time.

G3 - Insufficient Empirical Evaluation of Cognitive Load: While several papers propose AI-assisted desktop automation systems, rigorous empirical evaluation of cognitive load reduction using psychometric instruments (e.g., NASA-TLX) is largely absent from the literature. The subjective experience of users interacting with agentic desktop systems remains understudied.

G4 - Safety and Human-in-the-Loop Mechanisms for OS-Level Agents: As agentic systems gain the ability to execute irreversible OS-level actions (file deletion, email dispatch, application installation), the design of appropriate human-in-the-loop safety mechanisms becomes critical. This area remains inadequately addressed in current agentic AI research.

G5 - Proactive vs. Reactive Assistance in Desktop AI: The distinction between proactive assistance (anticipating user needs from context) and reactive command execution has not been rigorously evaluated in the context of desktop AI copilots. The conditions under which proactive assistance improves-rather than disrupts-user workflow are not well understood.

5. Research Questions

RQ1 [Intent Inference]: Can user intent be accurately inferred from natural language commands when augmented with real-time desktop context, compared to intent inference from natural language alone?

RQ2 [Contextual Retrieval Utility]: To what extent does the integration of multi-source desktop contextual information-spanning files, applications, browsing activity, and interaction history-improve task planning accuracy and task completion efficiency?

RQ3 [Cognitive Load Reduction]: Does interaction with an AI-powered context-aware desktop copilot measurably reduce user cognitive load, as assessed by the NASA Task Load Index, compared to conventional operating system interaction?

RQ4 [Agentic Task Planning Safety and Effectiveness]: Can autonomous multi-step task planning by a desktop AI agent improve user productivity while maintaining acceptable levels of user control, transparency, and operational safety?

6. Research Objectives

6. O1: To design and implement a multi-source desktop context retrieval system that aggregates file system metadata, open application state, browser activity, and interaction history into a unified, queryable knowledge representation.
7. O2: To develop a natural language intent understanding module that combines LLM-based semantic parsing with contextual retrieval to accurately classify user intent and extract actionable task specifications.
8. O3: To design and implement an agentic task planning engine capable of decomposing high-level user goals into executable sequences of desktop actions across multiple applications.
9. O4: To construct a persistent, privacy-preserving episodic memory system that retains user preferences, recurring workflow patterns, and interaction history across sessions.
10. O5: To implement a safety-conscious human-in-the-loop control mechanism that intercepts high-risk or irreversible actions and presents them for user approval before execution.
11. O6: To evaluate the system through controlled user studies measuring task completion time, interaction count, task success rate, and cognitive workload (NASA-TLX) against a baseline of conventional desktop interaction.
12. O7: To identify the conditions under which context-aware proactive assistance improves user productivity and to characterise failure modes and limitations of the proposed approach.

7. Proposed Methodology

7.1 Research Design

This project adopts a Design Science Research (DSR) methodology (Hevner et al., 2004), which provides a rigorous framework for the construction and evaluation of novel IT artefacts. The research proceeds through three iterative phases: (1) problem identification and literature review; (2) design, prototyping, and iterative refinement; and (3) empirical evaluation through controlled user studies.

7.2 Phase 1 - Literature Review and Requirements Analysis

This phase encompasses the following activities:

- Systematic review of literature on agentic AI systems, LLM-based task planning, RAG architectures, desktop automation, and human-computer interaction research from 2020 onwards.
- Comparative analysis of existing AI desktop assistants and copilot systems to identify functional gaps and design opportunities.
- Elicitation of functional and non-functional system requirements through analysis of representative desktop task scenarios.
- Identification and evaluation of candidate technology components, frameworks, and APIs.

7.3 Phase 2 - System Design and Prototyping

The system will be designed following a modular, layered architecture (see Section 8). Each module will be prototyped, unit-tested, and integrated incrementally. The development methodology will follow agile principles with two-week sprint cycles, enabling iterative refinement based on internal testing and supervisor feedback. Initial prototypes will focus on core functionality: intent understanding, context retrieval, and single-application task execution. Subsequent iterations will introduce multi-step planning, persistent memory, and the safety layer.

7.4 Phase 3 - Empirical Evaluation

A controlled user study will be conducted with a target sample of 20–30 participants recruited from the university community. Participants will complete a standardised set of representative desktop tasks under two conditions: (A) conventional OS interaction and (B) AEGIS-assisted interaction. Task scenarios will be drawn from real-world desktop workflows identified during the requirements analysis phase. Evaluation metrics are specified in Section 12. Ethical approval will be obtained from the university research ethics committee prior to participant recruitment.

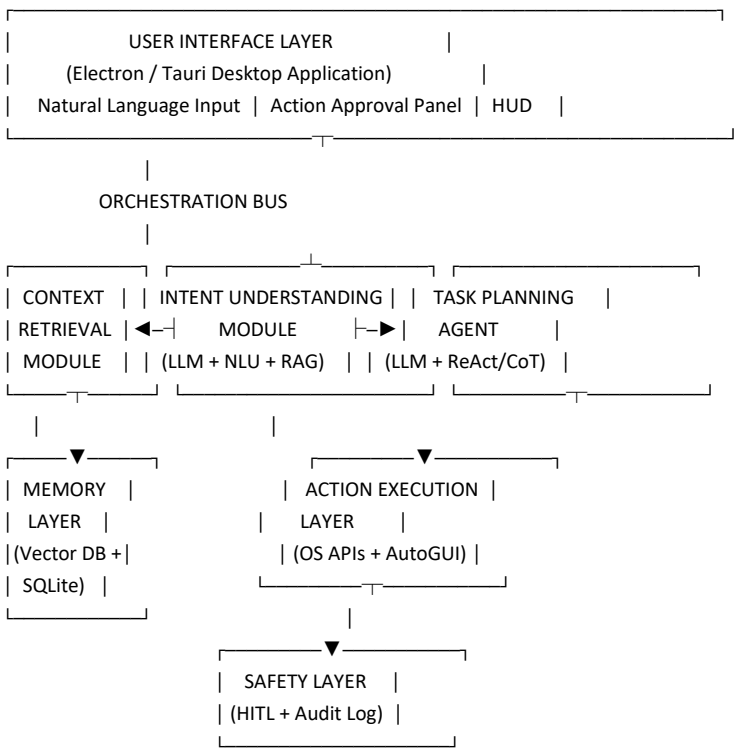
8. System Architecture

8.1 Architectural Overview

AEGIS is structured as a layered, event-driven architecture comprising six primary modules interconnected through a central orchestration bus. The architecture is designed for modularity,

extensibility, and clear separation of concerns between perception, reasoning, and actuation layers. Figure 1 provides a schematic representation of the system architecture.

[FIGURE 1 - AEGIS SYSTEM ARCHITECTURE DIAGRAM]



8.2 Module Descriptions

Module 1 - User Interface Layer

The user-facing component provides a non-intrusive desktop interface for natural language input, real-time status display, and action approval interaction. The interface is designed to minimise disruption to the user's primary workflow. It exposes a heads-up display (HUD) overlay, a command input panel, and an approval queue for safety-gated actions. The technology selection between Electron and Tauri will be determined during Phase 1 based on performance and integration requirements.

Module 2 - Intent Understanding Module

The Intent Understanding Module (IUM) receives natural language input from the user and produces a structured intent representation comprising an action class, target entities, parameters, and confidence scores. The IUM operates in two stages: (i) LLM-based semantic parsing, which interprets the user's utterance in isolation; and (ii) context-augmented intent refinement, which enriches the initial interpretation with contextual signals retrieved from the Context Retrieval Module (CRM). This two-stage approach allows the system to resolve ambiguous references (e.g., 'that report' → the most recently edited document) and infer implicit parameters (e.g., 'send the draft' → to the last-opened email recipient).

Module 3 - Context Retrieval Module

The CRM maintains a live, queryable index of the user's desktop environment. It ingests signals from four primary sources: (i) the file system, including file metadata, recent modifications, and content embeddings for searchable documents; (ii) running application state, including window titles, open document names, and application-level metadata; (iii) browser context, including open tab URLs, page titles, and browsing recency; and (iv) interaction history, including prior natural language exchanges, task executions, and their outcomes. Context is stored in a vector database to enable semantic similarity search and is augmented with a structured relational store for precise metadata queries.

Module 4 - Task Planning Agent

The Task Planning Agent (TPA) receives a structured intent specification and produces an executable action plan. The TPA employs an LLM-based planning loop using the ReAct (Reasoning and Acting) paradigm (Yao et al., 2023), in which the model alternates between reasoning steps and tool invocations. The TPA has access to a defined toolkit of desktop actions (see Module 5) and can compose these into multi-step plans. For complex, long-horizon tasks, hierarchical task decomposition will be employed, breaking high-level goals into sub-goals that are planned and executed independently. Plan execution is monitored, and the TPA can revise plans in response to execution failures or unexpected environmental changes.

Module 5 - Action Execution Layer

The Action Execution Layer (AEL) translates abstract action specifications produced by the TPA into concrete OS-level operations. The AEL exposes a standardised action API to the TPA and internally dispatches to the appropriate execution backend depending on action type. Supported action categories include: file system operations (open, create, move, rename, search); application control (launch, close, focus, send keystrokes); web browser control; system settings adjustment; and notification and reminder management. The AEL records the outcome of each action for logging and memory update purposes. The specific automation frameworks and APIs employed will be selected during Phase 1 based on compatibility and reliability assessments.

Module 6 - Memory Layer

The Memory Layer provides persistent, structured storage for three categories of user-specific knowledge: (i) episodic memory, encoding the history of user interactions and task executions with timestamps and contextual metadata; (ii) semantic memory, encoding learned user preferences, frequently used applications, recurring workflows, and named entity associations; and (iii) working memory, encoding the current task context and execution state within a session. Episodic and semantic memory are stored in a combination of a vector database (for semantic retrieval) and a relational database (for structured queries). Memory is updated after each interaction cycle and used by both the IUM and TPA during inference.

Module 7 - Safety Layer

The Safety Layer intercepts all action plans generated by the TPA before execution and applies a risk classification policy. Actions are classified into three risk tiers: (i) safe-executed automatically without user confirmation; (ii) cautionary-executed with a brief notification providing an undo

opportunity; and (iii) high-risk-requiring explicit user approval before execution. High-risk actions include irreversible file operations, outbound communications (email, messaging), and system configuration changes. The Safety Layer also maintains a comprehensive, human-readable audit log of all actions executed, approved, or rejected, supporting user accountability and system transparency.

9. Novel Contributions

This project advances the state of the art in AI-powered desktop assistance through the following novel contributions:

C1 - Context-Aware Desktop Intelligence Framework: A unified architectural framework that integrates multi-source desktop context retrieval-spanning the file system, running applications, browser state, and interaction history-with LLM-based intent understanding and agentic task planning. To the best of the authors' knowledge, no existing system combines all four context sources in a single, coherent retrieval architecture for desktop AI.

C2 - Two-Stage Context-Augmented Intent Understanding: A novel two-stage intent understanding pipeline in which initial LLM-based semantic parsing is subsequently refined using contextual signals retrieved from the live desktop environment. This approach enables disambiguation of implicit and underspecified user requests that cannot be resolved from natural language alone.

C3 - Persistent Cross-Session Episodic Memory for Desktop Agents: A memory architecture that accumulates user-specific episodic and semantic knowledge across interaction sessions, enabling personalised, increasingly tailored assistance without requiring explicit user configuration.

C4 - Human-in-the-Loop Safety Architecture for OS-Level Agentic Systems: A risk-tiered safety control mechanism designed specifically for the desktop automation context, balancing the efficiency benefits of autonomous action execution against the safety requirements of irreversible OS-level operations.

C5 - Empirical Cognitive Load Evaluation Methodology: A structured experimental methodology for evaluating the cognitive load impact of AI-powered desktop copilot systems using validated psychometric instruments, contributing to the empirical evidence base for human-AI interaction research in the desktop computing domain.

10. Expected Outcomes

10.1 Primary Deliverables

- A fully functional AEGIS desktop copilot prototype implementing all six architectural modules described in Section 8.
- A comprehensive system architecture and design document detailing all module specifications, data flows, and integration interfaces.

- An empirical evaluation report presenting quantitative and qualitative results from the controlled user study.
- A final project dissertation constituting the primary academic output of the project.

10.2 Anticipated Research Findings

- Context-augmented intent understanding will achieve meaningfully higher task specification accuracy than intent understanding from natural language alone, particularly for implicit or elliptical user requests.
- AEGIS will reduce mean task completion time and mean number of manual interactions for representative desktop task scenarios, compared to baseline conventional OS interaction.
- Participants will report statistically significantly lower cognitive workload scores (NASA-TLX) when using AEGIS for complex multi-step tasks.
- The human-in-the-loop safety mechanism will be rated favourably by users on control satisfaction scales, without introducing unacceptable friction for low-risk tasks.
- Persistent memory will improve task personalisation quality over time, as evidenced by reduced need for explicit parameter specification in repeat task scenarios.

11. Evaluation Methodology

11.1 Experimental Design

Evaluation will employ a within-subjects counterbalanced experimental design. Each participant will complete a standardised task battery under two conditions: (A) Conventional OS Interaction (baseline) and (B) AEGIS-Assisted Interaction. Condition order will be counterbalanced across participants to mitigate learning effects. A washout period will separate the two conditions. The task battery will comprise 8–10 representative desktop task scenarios of varying complexity, derived from real-world usage patterns identified during requirements analysis.

11.2 Participants

A target sample of 20–30 participants will be recruited from the university community. Inclusion criteria will require regular use of a personal computer for work or study purposes. Participants will be screened for prior experience with AI assistants to enable subgroup analysis. All participants will provide informed consent prior to study commencement. Ethical approval will be obtained from the institutional research ethics board.

11.3 Evaluation Metrics

Metric	Measurement Instrument	Addresses RQ
Task Completion Time	Automated task timing (seconds)	RQ3, RQ4
Number of Manual Interactions	Automated interaction logging (clicks, keystrokes)	RQ3, RQ4
Task Success Rate	Binary task completion assessment by experimenter	RQ4
Cognitive Workload	NASA Task Load Index (NASA-TLX) questionnaire	RQ3

Intent Interpretation Accuracy	Expert annotation of system intent logs vs. user intent reports	RQ1
User Satisfaction	System Usability Scale (SUS) + semi-structured interview	RQ3, RQ4
Context Retrieval Relevance	Expert annotation of retrieved context logs vs. task requirements	RQ2
Safety Mechanism Acceptance	Custom Likert-scale questionnaire	RQ4

11.4 Analysis Approach

Quantitative data will be analysed using appropriate statistical tests (paired t-tests or Wilcoxon signed-rank tests depending on normality). Effect sizes will be reported using Cohen's d. Qualitative data from semi-structured interviews will be analysed using thematic analysis. Statistical significance will be assessed at the $\alpha = 0.05$ level.

12. Preliminary Technology Stack

The following technology stack is preliminary and subject to revision following the literature review, supervisor consultation, and feasibility assessment. It is presented to demonstrate the technical feasibility of the proposed system, not as a finalised implementation commitment. Final technology selection will be determined during the design and implementation phases.

Layer	Candidate Technologies	Notes
Frontend / UI	Electron, Tauri	Selection based on performance and platform integration requirements
Backend	Python, Node.js	Python preferred for AI/ML component integration
Large Language Models	OpenAI GPT-4o, Claude 3.5+, Gemini, Llama 3, Mistral	Cloud or local inference; selection based on cost, latency, and capability
Agent Framework	LangChain, LangGraph, AutoGen, CrewAI, custom	Selection based on planning capability and tool-use flexibility
RAG / Context Retrieval	LlamaIndex, LangChain RAG, custom pipeline	Integrated with vector database for semantic retrieval
Vector Database	ChromaDB, Weaviate, Qdrant, Pinecone	For semantic search over context embeddings
Relational Database	SQLite, PostgreSQL	For structured metadata, episodic memory, and audit logs
Desktop Automation	PyAutoGUI, pywinauto, Win32 API, WinRT	Platform-specific automation; cross-platform fallback via PyAutoGUI
NLP / Embedding	Sentence-Transformers, OpenAI	For document and context

	Embeddings, FastEmbed	embedding generation
OS Context APIs	Windows Management Instrumentation (WMI), psutil, win32com	For process monitoring, file system indexing

13. Project Timeline

Phase	Activities	Duration	Milestone
Phase 1 Literature Review	Systematic literature review; gap analysis; requirements elicitation; technology evaluation	Weeks 1–6	Requirements specification document; literature review chapter
Phase 2 System Design	Detailed architecture design; module specification; database schema design; UI/UX wireframing	Weeks 7–10	System design document; architecture diagrams
Phase 3 Core Development	IUM, CRM, and Memory Layer implementation; unit testing; integration testing	Weeks 11–18	Working prototype (IUM + CRM + Memory)
Phase 4 Agent Development	TPA, AEL, and Safety Layer implementation; end-to-end integration; system testing	Weeks 19–24	Complete AEGIS prototype; integration test report
Phase 5 Evaluation	User study design; ethics approval; participant recruitment; controlled user study; data collection	Weeks 25–30	Evaluation dataset; raw results
Phase 6 Analysis and Write-up	Statistical analysis; results interpretation; dissertation writing; revision; submission	Weeks 31–36	Final dissertation; project poster; demo video

14. Limitations, Ethical Considerations, and Future Work

14.1 Limitations

- Platform Scope: The initial prototype will target the Windows operating system. Cross-platform generalisation to macOS and Linux is a subject for future work.
- LLM Dependency: The system's planning and intent understanding capabilities are contingent on the performance of the underlying LLM. Hallucinations, errors in tool selection, and context window limitations may affect system reliability.
- Sample Size: The planned user study sample (20–30 participants) may limit the statistical power of quantitative findings. Replication with larger, more diverse populations will be required to establish broad generalisability.
- Task Representativeness: The standardised task battery may not fully capture the diversity of real-world desktop workflows. Ecological validity will be carefully considered in task design.

- Latency: LLM inference latency may affect perceived system responsiveness for time-sensitive tasks.

14.2 Ethical Considerations and Privacy

AEGIS operates in close proximity to sensitive user data, including personal documents, browsing history, and application content. The following ethical commitments will guide system design and evaluation:

- Data Minimisation: Context retrieval will be strictly scoped to information necessary for task planning. The system will not transmit personal data to external services without explicit user consent.
- Local Processing Priority: Where technically feasible, AI inference will be performed locally or via on-device models to minimise data exposure. Cloud LLM usage will be limited to non-sensitive contexts or will be subject to data anonymisation.
- Informed Consent: All study participants will be provided with clear, jargon-free explanations of what data is collected, how it is used, and their right to withdraw.
- Transparency and Explainability: The system will provide human-readable explanations of planned actions prior to execution, supporting user understanding and informed approval.
- Safety by Design: The risk-tiered safety mechanism is a first-class architectural concern, not an afterthought. Irreversible actions will always require explicit user confirmation.
- Audit and Accountability: All executed actions will be logged in a user-accessible audit trail.

14.3 Future Work

- Extension to macOS and Linux platforms.
- Integration of multimodal input (voice, screen understanding via vision-language models).
- Development of a fine-tuned intent classification model trained on desktop interaction corpora.
- Longitudinal study of memory-driven personalisation effects over extended usage periods.
- Exploration of multi-agent collaboration patterns for complex, parallel desktop workflows.
- Investigation of federated or differential privacy mechanisms for the Memory Layer.

15. References

13. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
14. Chase, H. (2022). LangChain: Building applications with LLMs through composability. GitHub Repository. https://github.com/langchain-ai/langchain
15. Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., ... & Su, Y. (2023). Mind2Web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36.
16. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
17. González, V. M., & Mark, G. (2004). 'Constant, constant, multi-tasking craziness': Managing multiple working spheres. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 113–120). ACM.
18. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

19. Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., ... & Tang, J. (2024). CogAgent: A visual language model for GUI agents. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 14281–14290).
20. Hu, Y., Shi, H., Wang, L., Hwang, J., Courville, A., Kumar, A., ... & Neubig, G. (2024). AgentBench: Evaluating LLMs as agents. *International Conference on Learning Representations*.
21. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
22. Li, X., Liu, H., & Yang, J. (2023). UFO: A UI-focused agent for Windows OS interaction. *arXiv preprint arXiv:2402.07939*.
23. Mark, G., Iqbal, S., Czerwinski, M., & Johns, P. (2014). Capturing the mood: Facebook and face-to-face encounters in the workplace. *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work & Social Computing* (pp. 1082–1094). ACM.
24. Nakano, R., Hilton, J., Balwit, A., Wu, J., Ouyang, L., Kim, C., ... & Schulman, J. (2022). WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.
25. Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (pp. 1–22). ACM.
26. Rawles, C., Li, A., Rodriguez, D., Riva, O., & Baldridge, J. (2023). AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*.
27. Shi, T., Karpathy, A., Fan, L., Hernandez, J., & Liang, P. (2017). World of bits: An open-domain platform for web-based agents. *Proceedings of the 34th International Conference on Machine Learning*, 70, 3135–3144.
28. Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., ... & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
29. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... & Wen, J. R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
30. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., ... & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*.
31. Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., ... & Yu, T. (2024). OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*.
32. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.
33. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.
34. Zhang, C., Yang, Z., Liu, J., Han, S., Chen, J., Gao, H., ... & Li, Q. (2023). AppAgent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*.
35. Zhu, X., Chen, Y., Tian, H., Tao, C., Su, W., Yang, C., ... & Dai, J. (2023). Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*.